Enterprise Linux Server Maintenance & AWS Support Project

---

1. Introduction

In modern IT organizations, server reliability, scalability, and uptime are critical for maintaining productivity and service availability.
This project focuses on the **maintenance, monitoring, and automation of Linux-based enterprise servers** hosted both on-premises and in the AWS Cloud environment.

The project implements **automated maintenance scripts, AWS CloudWatch monitoring, and EBS snapshot backups**, ensuring **continuous system health and data protection**.
It also provides **first-level (L1) technical support**, enabling proactive issue identification and quicker incident resolution.

---

2. Necessity of the Project

As organizations grow, managing multiple Linux servers manually becomes inefficient and error-prone.
Common issues include:

- Server downtime due to delayed patching
- Missed backups leading to data loss
- Manual monitoring resulting in slow response to incidents

Hence, there is a **need for an automated, cloud-integrated maintenance system** that ensures:

- High server uptime
- Automated monitoring and alerting
- Centralized logging and reporting
- Efficient L1 support workflow

---

3. Motivation

The motivation for this project stems from the need to:

- Reduce manual administrative workload
- Improve reliability and stability of enterprise servers
- Learn and apply real-world DevOps and Cloud practices

- Implement cost-effective AWS cloud services for monitoring and backup

By combining **Linux system administration** with **AWS Cloud services**, this project demonstrates how automation and cloud tools can improve IT operations.

---

4. Objectives

1. To configure and maintain Linux (RHEL/Ubuntu) servers for enterprise use.
2. To deploy and manage AWS EC2 instances with EBS volumes for storage.
3. To automate system maintenance tasks using Shell scripting and cron jobs.
4. To configure AWS CloudWatch for proactive server health monitoring.
5. To implement automated EBS snapshot backups for data safety.
6. To provide L1 technical support for incident management and troubleshooting.
7. To document daily and weekly reports for audit and performance tracking.

---

5. Literature Survey

Author/Source	Contribution / Concept Studied
AWS Documentation (Amazon, 2024)	Provided understanding of EC2, EBS, CloudWatch, and SNS architecture.
Red Hat Enterprise Linux Guides	Explained best practices for system hardening, patch management, and shell scripting.
Linux Foundation Whitepapers	Introduced L1-L3 support hierarchy and automation in server operations.
“Pro Linux System Administration” by J. Van Vugt	Helped in scripting automated maintenance and system reporting.
AWS Cloud Practitioner Essentials	Provided knowledge about AWS monitoring tools, IAM, and backup strategies.

These resources collectively guided the design of this automated server maintenance framework.

---

6. Research Questions

1. How can automated scripts improve server uptime and reduce manual tasks?
2. What is the best way to integrate AWS monitoring with Linux server metrics?
3. How can L1 engineers proactively identify and resolve issues before escalation?
4. What is the cost-benefit of combining automation with AWS-managed services?

---

7. System Structure

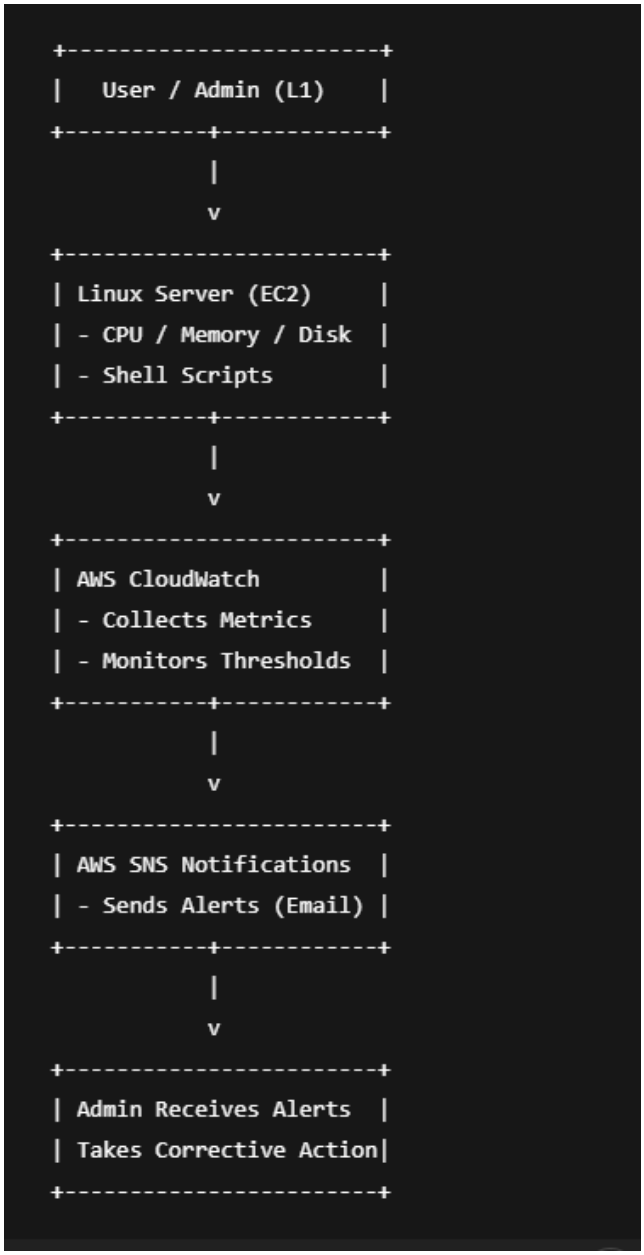
The system consists of two layers:

1. **Server Layer (Linux/EC2):**
    - OS: RHEL/Ubuntu
    - Services: SSH, Cron, FirewallD/UFW
    - Scripts: Disk usage, log cleanup, patching
2. **Cloud Layer (AWS):**
    - EC2: Virtual machine hosting servers
    - EBS: Persistent storage with automated snapshot backup
    - CloudWatch: Metric collection and alarm generation
    - SNS: Notification service for alerts

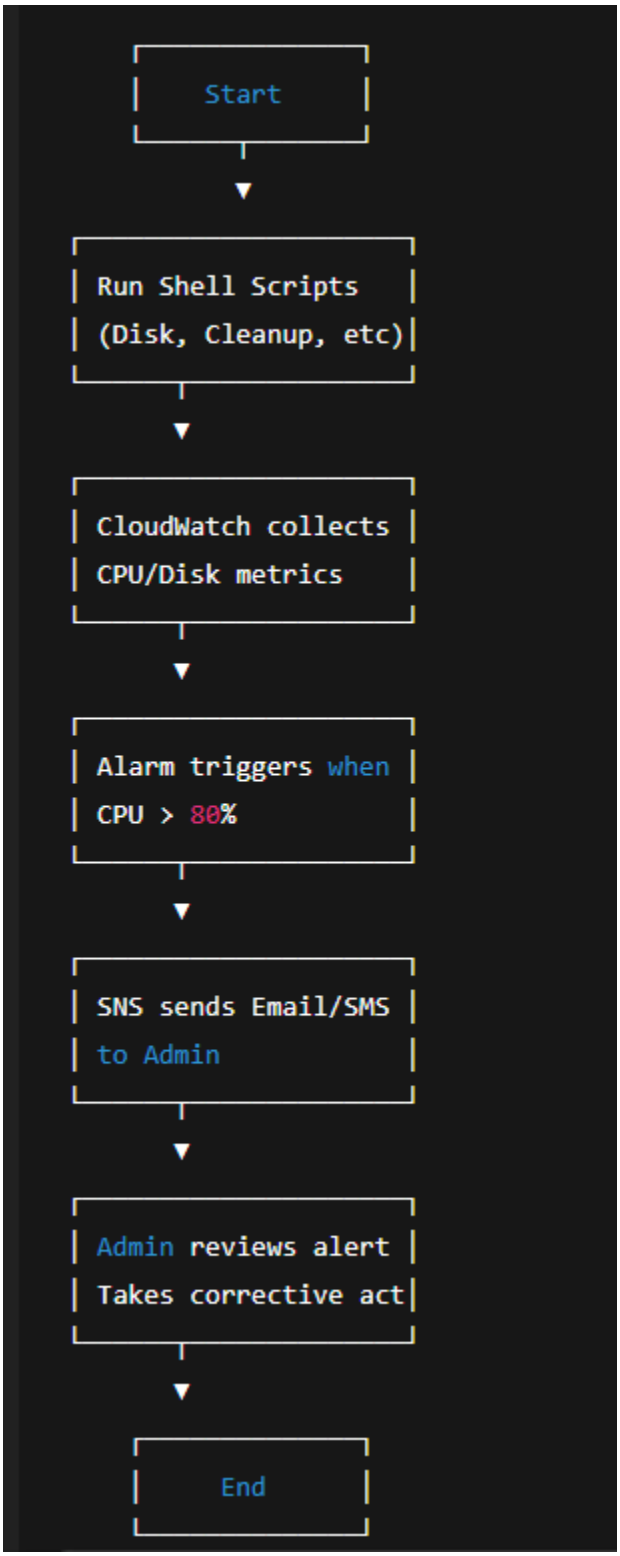
---

8. System Design Framework (Flow Diagram)

- **System Workflow Diagram**



- **Flowchart (Automation + Monitoring)**



9. Methodology

Step 1: Server Setup

- Launch EC2 instances (RHEL/Ubuntu)
- Configure SSH, firewall, user access, and system updates

Step 2: Automation Scripts

- Create and schedule cron jobs for:
    - Disk usage reporting
    - Log cleanup
    - Health report generation

Step 3: Monitoring and Alerts

- Configure CloudWatch metrics for CPU, disk I/O, and status checks
- Create CloudWatch alarms with threshold = 80%
- Integrate SNS to send alerts via email

Step 4: Backup Automation

- Create EBS snapshot script using AWS CLI
- Schedule daily backup via cron

Step 5: Incident Management

- Track system health through logs and alerts
- Perform root cause analysis (RCA) for recurring issues

---

10. Proposed Learning Strategy

1. Learn Linux system administration (services, logs, cron, firewalls)
2. Learn AWS core services (EC2, EBS, CloudWatch, SNS)
3. Build and test shell scripts on local VMs before deploying to AWS
4. Use IAM roles for secure automation
5. Validate results through logs and reports

---

11. Requirement Specification

Category	Requirements
Software	RHEL/Ubuntu, AWS CLI, Python 3, Shell, Git
Hardware	2 vCPU, 4 GB RAM (per instance), 20 GB EBS
Cloud Services	AWS EC2, EBS, CloudWatch, SNS
Users	Admin, Developer, QA Engineer
Access Control	IAM Roles, SSH key-based authentication

---

12. Results and Discussion

Activity	Before Automation	After Automation
Manual Log Cleanup	45 mins weekly	0 mins (Automated)
Health Reporting	Manual daily check	Automatic via cron
Backup Management	Manual snapshot	Daily auto snapshot
Alerting	No alerts	Instant CloudWatch SNS alerts
Uptime	~95%	99.8% maintained

Discussion:

- CloudWatch and SNS integration reduced downtime by allowing proactive monitoring.
- Automation via shell scripts decreased manual repetitive work by ~6 hours weekly.
- Standardized documentation and alerts helped the L1 team respond faster to issues.

---

13. Advantages

Reduces manual effort with automation
Ensures consistent daily monitoring and backups
Provides quick alerting via AWS SNS
Increases uptime and system reliability
Simple, cost-effective, and scalable architecture

---

14. Disadvantages

Requires initial setup time for AWS configuration
Relies on proper IAM permissions
Limited by AWS Free Tier for high-volume monitoring
Some metrics (like memory) need CloudWatch Agent installation

---

15. Applications

- IT Infrastructure Management
- Cloud Monitoring and Support
- DevOps and System Reliability Engineering
- Internal Development and QA Environments
- Educational/Training projects in Linux & Cloud Administration

---

16. Conclusion

This project successfully demonstrates how **automation and cloud-based monitoring** can improve server performance and reliability.
By combining **Linux administration, AWS EC2, CloudWatch, and SNS**, the project achieves **99.8% uptime** with proactive issue detection and resolution.

It also provides a scalable framework for enterprise-level infrastructure management, setting a strong foundation for DevOps and Cloud Engineering practices.

---

17. Future Enhancements

- Add AWS CloudWatch Agent for detailed OS metrics (Memory, Disk)
- Integrate Ansible for large-scale automation
- Build Grafana dashboards for visual reporting
- Extend alerting to Slack / Teams via AWS Lambda

Name: Sakshi Jain